Cây nhị phân và cây tìm kiếm nhị phân

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 4: Tra cứu và dữ liệu có thứ tự.
- Các bài trong chương: Bảng băm, tập hợp và ánh xạ; Cây nhị phân và cây tìm kiếm nhị phân.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Phân biệt cây nhị phân với BST; mô phỏng tìm kiếm và duyệt inorder.

Kiến thức nền

Cây có các nút và quan hệ cha–con. Cây nhị phân có tối đa hai con tại mỗi nút. BST bổ sung quy tắc thứ tự; trong phiên bản không có khóa trùng, mọi khóa bên trái nhỏ hơn nút và bên phải lớn hơn nút. Có hai con không đủ để kết luận là BST.

Tìm kiếm so sánh mục tiêu với khóa nút hiện tại rồi chọn một nhánh. Chi phí phụ thuộc chiều cao h, thường O(h). Nếu cây cân đối thì h là O(log n); nếu chèn khóa tăng dần vào BST không tự cân bằng, cây có thể lệch và tìm kiếm tới O(n).

Duyệt inorder đi trái, nút, phải; với BST hợp lệ cho dãy tăng dần. Kiểm tra BST phải xét ràng buộc toàn cây con, không chỉ hai nút con trực tiếp. Bài này giới thiệu nguyên lý; thuật toán tự cân bằng như AVL không thuộc yêu cầu thực hành bắt buộc.

1. Các khái niệm cần thống nhất

Gốc không có cha; lá không có con. Chiều cao có thể tính bằng số cạnh hoặc số nút tùy tài liệu; phải ghi quy ước. Bài này dùng chiều cao theo số cạnh khi mô tả sơ đồ, còn phân tích O(h) không thay đổi ý nghĩa tiệm cận.

2. Tìm kiếm BST không dùng đệ quy

```
class Node:
    def __init__(self, key, left=None, right=None):
        self.key, self.left, self.right = key, left, right

def contains(root, target):
    node = root
```

```
    while node is not None:
        if target == node.key:
            return True
        node = node.left if target < node.key else node.right
    return False

root = Node(4, Node(2, Node(1), Node(3)), Node(6))
assert contains(root, 3)
assert not contains(root, 5)
```

Thuật toán dùng bộ nhớ phụ O(1) cho tìm kiếm lặp; chi phí thời gian phụ thuộc chiều cao. Không lấy chiều cao của một hình cân đối để kết luận mọi cây đều log n.

3. Kiểm tra thứ tự toàn cây

Gốc 10, con trái 5, nhưng trong cây con trái có 12: mỗi cặp cha-con cục bộ có thể trông hợp lý ở một số cạnh nhưng 12 vẫn vi phạm ràng buộc nhỏ hơn 10. Kiểm tra hợp lệ cần truyền khoảng cho toàn cây con hoặc xem inorder theo chính sách khóa trùng.

4. Bài tập có hướng giải

Thêm thao tác chèn không cho khóa trùng; khi gặp bằng thì giữ nguyên hoặc báo lỗi theo hợp đồng. Viết test cây rỗng, khóa ở gốc, hai nhánh và chuỗi lệch. So sánh thứ tự chèn tăng dần với thứ tự tạo cây cân đối; không thay đổi bộ dữ liệu giữa hai phép so sánh.

> **Bài tập vận dụng bổ sung**
>
> Vẽ BST khi chèn 1,2,3,4 và so sánh chiều cao với thứ tự 3,1,4,2. Viết truy vết tìm một khóa không tồn tại.

Hướng giải bài tập bổ sung

Cây chèn tăng dần có thể thành chuỗi. Không ghi mọi thao tác BST là O(log n) nếu chưa có điều kiện về chiều cao.

Câu hỏi tự kiểm tra

1. Cây nhị phân có đặc điểm nào?
2. Trong BST không khóa trùng, khóa ở cây con trái so với nút thế nào?
3. Inorder của BST hợp lệ cho thứ tự nào?
4. Tìm kiếm BST có chi phí phụ thuộc gì?
5. Chèn tăng dần vào BST không tự cân bằng có thể gây gì?

Gợi ý đáp án và điểm cần nhớ

- Mỗi nút tối đa hai con. Giới hạn số con không tự tạo quy tắc tìm kiếm.

- Nhỏ hơn khóa nút. Quy tắc thứ tự áp dụng cho toàn cây con.
- Tăng dần. Inorder là trái–nút–phải.
- Chiều cao cây. Mỗi bước đi xuống một mức.
- Cây lệch và tìm kiếm O(n). Trường hợp xấu cây có chiều cao tỉ lệ n.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html